

DAYTONE MOOD ANALYSER

A comprehensive mental wellness tracking platform utilizing local NLP sentiment analysis and machine learning-based burnout classification.

Candidate Name:	S A SANUSH
Academic Project:	Semester 3 Mini Project
Course Program:	Master of Computer Applications (MCA)
Sponsoring Body:	ICT Academy of Kerala
Submission Date:	May / October 2026
Core Stack:	Python, Flask, SQLite, Scikit-learn, VADER NLP, Three.js WebGL

LIST OF FIGURES

Figure No.	Title / Description
Figure 1	Home page of the DayTone Mood Analyser web application
Figure 2	Proposed system architecture of DayTone platform
Figure 3	Relational SQLite database schema diagram
Figure 4	User registration and secure login screen layout
Figure 5	Daily check-in and encrypted journal submission form
Figure 6	User dashboard showing mood trends and WebGL 3D orb
Figure 7	Administrative interface displaying user catalogs and audit logs
Figure 8	Criteria-wise wellness score breakdown and suggested tips
Figure 9	Model diagnostics, bias reports and Kolmogorov-Smirnov drift charts

TABLE OF CONTENTS

Chapter	Description	Page No.
1	Problem Definition (Overview & Problem Statement)	6
2	Introduction (Background, Objectives & Scope)	9
3	Literature Survey (Review of Techniques & Pipelines)	12
4	System Analysis (Existing, Proposed, Requirements)	15
5	System Design (Architecture, DB & Modules)	19
6	Implementation (Tech Stack, Shaders, ML/NLP)	24
7	Result & Discussion (Outputs, Drift & Diagnostics)	36
8	Conclusion & Future Enhancements	39
App A	Appendix A: Project File Structure	40
App B	Appendix B: Important Routes	41
App C	Appendix C: Important Functions	42
App D	Appendix D: Requirements	43
App E	Appendix E: Important Code Snippets	44
Ref	References & Bibliography	51

LIST OF ABBREVIATIONS

Abbreviation	Expansion / Full Form
AI	Artificial Intelligence
NLP	Natural Language Processing
ML	Machine Learning
PWA	Progressive Web Application
CSRF	Cross-Site Request Forgery
AES	Advanced Encryption Standard
SOC	System and Organization Controls
GDPR	General Data Protection Regulation
CSP	Content Security Policy
XAI	Explainable Artificial Intelligence

ABSTRACT

The **DayTone Mood Analyser (DMA)** is a self-hosted, privacy-first, and GDPR-compliant web application developed using Python, Flask, SQLAlchemy, and Scikit-learn. The system acts as a secure personal logging tracker and analytics engine, enabling users to register their daily wellness indicators (mood score, sleep hours, stress level, activity completion, and social interaction metrics) alongside reflective journal entries. The objective of the platform is to leverage local Natural Language Processing (NLP) and Machine Learning (ML) techniques to evaluate mental fatigue, assess cumulative burnout risks, and output explainable wellness recommendations without compromising sensitive user data.

To protect data at rest, DayTone implements hybrid column-level symmetric encryption (AES-256 Fernet) for raw text reflection notes. Textual analyses are run locally using the NLTK VADER sentiment lexicon with an optional transformer-based DistilBERT pipeline fallback. Burnout risks are evaluated through a 12-feature mathematical matrix processed by a Decision Tree or Random Forest classifier. Diagnostic utilities, including model bias audits and prediction drift monitors, are integrated directly into the administrative console. Additionally, the platform functions as a Progressive Web Application (PWA) supporting offline fallbacks, multi-tab synchronization, query caching, and WCAG 2.1 motion accessibility guidelines (Calm Mode).

1. PROBLEM DEFINITION

1.1 OVERVIEW

In modern professional and academic settings, chronic stress, mental fatigue, and clinical burnout have become widespread. Under the pressure of corporate deliverables, university placement exams, and long working hours, individuals often experience progressive declines in physical and emotional well-being without clear warnings. Traditional healthcare systems operate reactively, treating clinical exhaustion only after severe symptoms manifest. Preventive systems—specifically self-tracking wellness platforms—provide a proactive solution, allowing users to log behavioral patterns and identify early signs of burnout before they escalate.

However, current wellness applications present structural drawbacks. Most commercial trackers require users to upload highly sensitive, personal journal logs to proprietary third-party clouds where data is processed by black-box algorithms. This practice raises privacy concerns and exposes user data to unauthorized leaks or advertising profiles. In high-performance developer environments and academic institutions, there is a clear need for a self-hosted, local-first alternative that processes data privately while maintaining the analytical depth of modern machine learning pipelines.

1.1 OVERVIEW (CONTINUED)

Beyond standard data security, mental wellness trackers must also address technical and operational challenges. Relational database backends must store journals securely, preventing unauthorized read access even if the database file is compromised. Furthermore, applications must be resilient to connectivity failures. Under high-latency network conditions, or when users travel, web-dependent platforms fail. This requires offline-first support using local client-side caches and service workers.

Additionally, algorithmic transparency (Explainable AI or XAI) is critical in mental health tracking. If a platform flags a user as having a 'High' risk of burnout, it must explain the specific factors driving that prediction (such as a drop in sleep hours or a spike in stress levels). Providing a single, unexplained risk level can cause user anxiety. DayTone addresses these requirements by building a secure, local-first, and explainable wellness tracking platform.

1.2 PROBLEM STATEMENT

The objective of this project is to design and implement a self-hosted, secure, and transparent wellness tracker that evaluates daily physical and mental metrics locally. The system must accept user inputs (mood, sleep, stress, activity, and social indicators), extract textual features from journal notes privately (using local sentiment libraries), and evaluate cumulative burnout risks (Low, Medium, High) via an explainable machine learning model. To comply with privacy standards (GDPR), the system must encrypt reflections at rest, provide account purge mechanisms (Right to be Forgotten), protect interfaces against brute-force requests, and operate offline as a Progressive Web App.

[SCREENSHOT PLACEHOLDER]
Figure 1: Home Page of DayTone Mood Analyser UI
(Insert User Interface Screen Here)

Figure 1: Home page of the DayTone Mood Analyser

2. INTRODUCTION

2.1 BACKGROUND OF THE PROJECT

Mental health tracking systems have historically transitioned from physical paper journals to digital databases. However, raw logging is rarely sufficient to trigger user self-awareness. Incorporating data science and machine learning allows trackers to detect trends, predict risks, and make recommendations. The background of this project lies in creating a student-friendly, free-tier deployable system that implements security (AES-256), data compliance (GDPR), accessibility (WCAG), and AI modeling comparisons (Decision Tree vs. Random Forest). This approach provides a practical tool for academic and personal use.

Furthermore, by structuring this as a self-contained Python Flask project, we establish a template for local-first mental health software. The project highlights that advanced NLP and machine learning tasks do not require expensive cloud infrastructure. Lightweight model payloads (such as joblib-serialized scikit-learn classifiers and rule-based lexicons) can run directly on CPU-constrained servers, making secure AI analytics more accessible.

2.2 OBJECTIVES OF THE PROJECT

The primary design, implementation, and deployment objectives of DayTone are:

- **Local NLP Processing:** Use VADER or DistilBERT models locally in Flask memory to compute journal sentiment score indexes without cloud network dependence.
- **ML Risk Classification:** Implement a 12-feature scikit-learn classifier model to predict burnout risk levels, comparing Decision Tree and Random Forest algorithms.
- **Data Confidentiality:** Secure database columns using cryptography Fernet structures (AES-256) to ensure sensitive notes are encrypted at rest.
- **GDPR Compliance:** Establish data portability exports (CSV/PDF) and user purging cascades (Right to be Forgotten).
- **PWA Implementations:** Precache core assets to allow offline use, sync multi-tab page states via BroadcastChannels, and support motion accessibility (Calm Mode).

2.3 SCOPE OF THE PROJECT

The project scope focuses on preliminary screening, sentiment analysis, and wellness recommendations. It does not serve as a clinical diagnostic tool, make medical decisions, or replace professional therapy. It is an educational and self-monitoring dashboard meant to identify mental fatigue trends.

2.4 METHODOLOGY

The software development methodology follows an iterative, component-driven approach. First, database schemas and SQLAlchemy models are defined. Second, user log routes, validation, and encryption decorators are implemented. Third, the local sentiment parser and ML inference functions are integrated into check-in forms. Fourth, the dashboard charts (Chart.js) and 3D WebGL orb (Three.js) are built. Finally, drift detection, bias auditing, and rate limit protections are implemented for production deployment.

3. LITERATURE SURVEY

3.1 REVIEW OF WELL-BEING TRACKING TECHNIQUES

Early digital mood trackers recorded inputs in relational tables and presented simple line charts. These designs were limited because they relied solely on manual inputs and ignored the qualitative insights within text journals. Literature reviews show that text reflections contain emotional signals that may contradict simple quantitative scores. Modern trackers use hybrid architectures that combine quantitative check-ins with natural language processing of qualitative journals.

Additionally, early designs lacked structured database encryption at rest. If the backend SQL engine was compromised, sensitive personal records were exposed. This vulnerability highlights the importance of implementing transparent column-level encryption on text fields. Modern designs also emphasize local processing to minimize data transmission risks.

3.2 REVIEW OF NLP IN SENTIMENT ANALYSIS

Sentiment analysis generally utilizes rule-based lexicons (e.g. NLTK VADER) or deep learning transformer models (e.g. BERT variants). VADER is lightweight and computationally efficient on standard CPUs, making it suitable for self-hosted platforms. However, it lacks context-awareness and struggles with sarcasm. Transformer models, such as DistilBERT, evaluate semantic relationships and multi-word negations but require additional memory and execution overhead. A hybrid fallback model balances execution speed with contextual accuracy.

In addition, clinical research indicates that VADER may miss domain-specific distress terms. While BERT models provide better accuracy, their resource footprints make them difficult to run on free-tier servers. DayTone implements a fallback mechanism, prioritizing VADER for speed and low memory while supporting DistilBERT when hardware resources are available.

3.3 REVIEW OF MACHINE LEARNING IN BURNOUT CLASSIFICATION

Burnout and fatigue classification research highlights the importance of historical context. Evaluating a single log entry lacks temporal depth. Modern classifiers (such as Decision Trees or Random Forests) construct features that capture averages, standard deviations, and consecutive indicators over a rolling time window (e.g. 7 days). This allows models to identify progressive changes in sleep, stress, and mood, leading to higher classification accuracy.

Additionally, studies show that simple Decision Trees are prone to overfitting on synthetic training data. Random Forest classifiers mitigate this by building ensembles of decision trees, which reduces variance and improves generalization. To maintain transparency, models must be combined with explainable AI (XAI) features that display the specific drivers behind predictions.

4. SYSTEM ANALYSIS

4.1 EXISTING SYSTEM

The existing system in many academic institutions and high-performance environments is either non-existent or reliant on manual Excel logging. Where web-based trackers are used, they typically upload raw data directly to remote cloud services. These commercial cloud applications do not process data locally. This results in high network dependence and exposes user data to cloud breaches or tracking by advertisers.

Counselors and managers who review these logs must manually check entries, which is slow, inconsistent, and subjective. There is no automated, transparent system to identify trends, check keywords, or flag high-risk users. This lack of automation delay feedback and limits the effectiveness of preventive mental health support.

4.2 LIMITATIONS OF EXISTING SYSTEM

Key limitations identified in cloud-dependent mood trackers include:

- **Lack of Privacy:** Personal journals are processed by external third-party models, violating user confidentiality.
- **Vulnerable Storage:** Databases store notes in plaintext, leaving them open to exposure during data breaches.
- **Poor Accessibility:** Missing features like motion controls for users with visual or motion sensitivities.
- **No Trend Assessment:** Systems evaluate single check-ins in isolation, failing to assess historical trends.
- **High Overhead Costs:** Operating complex server stacks incurs continuous subscription fees.

4.3 PROPOSED SYSTEM

The proposed system, DayTone, addresses these issues through local execution, hybrid encryption, and offline capabilities. By utilizing local NLP sentiment scoring (VADER) and a local ML classifier, it evaluates user burnout risk locally. All journal entries are encrypted using AES-256 at rest, protecting user privacy in the SQLite database. Additional safety reveals shield high-risk warnings to manage user anxiety, and a Calm Mode setting accommodates users with motion sensitivities.

For administrators and counselors, DayTone provides an admin dashboard. Admins can review platform-wide metrics, track model performance, verify accuracy, and check for bias or drift. This design provides an explainable and repeatable self-monitoring dashboard.

4.4 FEASIBILITY STUDY

- **Technical Feasibility:** Strong. Stable open-source Python libraries (scikit-learn, VADER) ensure core logic is highly responsive and runs locally on standard CPU hosts.
- **Operational Feasibility:** High. Browser-based client requires no local setup, providing intuitive dark-themed inputs and charts for students.
- **Economic Feasibility:** Excellent. Open-source stacks eliminate licensing fees, allowing easy deployment on student-friendly free-tier hosting.

4.5 FUNCTIONAL REQUIREMENTS

The platform must support user authentication, daily log entry forms, local sentiment scoring, burnout predictions, goal progression, PDF/CSV downloads, and administrative bias/drift diagnostics.

4.6 NON-FUNCTIONAL REQUIREMENTS

Must achieve low response latencies (<1s), strict security (AES-256 column encryption, Talisman CSP headers), robust brute-force protection (Redis-backed rate limiting), and offline reliability (PWA cached assets).

5. SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE

The platform architecture is designed around a three-tier model, separating the browser front-end, Flask web controllers, and SQLite/PostgreSQL/Redis databases.

[SCREENSHOT PLACEHOLDER]
Figure 2: Proposed System Architecture of DayTone
(Insert User Interface Screen Here)

Figure 2: Proposed system architecture of the DayTone Mood Analyser

5.2 DATA FLOW DIAGRAM DESCRIPTION

The data flow in DayTone follows a defined, secure path: The user browser submits check-in metrics and journal text over HTTPS. The Flask validation layer sanitizes the inputs and computes the VADER sentiment score index locally. The system then builds the 12-feature matrix (incorporating rolling averages from SQLite history) and runs the machine learning predictor. The journal note is encrypted via AES-256 before the complete MoodLog record is written to the SQLite database, returning rendered JSON/HTML and updating the dashboard.

5.3 DATABASE DESIGN

The SQLite schema contains tables for users, profiles, logs, burnout history, suggestions, and audit logs. Foreign key constraints enforce referential integrity and support cascading purges.

[SCREENSHOT PLACEHOLDER]
Figure 3: SQLite Database Schema used for DayTone
(Insert User Interface Screen Here)

Figure 3: Relational SQLite database schema diagram

5.4 USER INTERFACE DESIGN

The interface design prioritizes accessibility, using a clean dark theme, clear tab-focus rings, and motion control toggles. It uses custom CSS grids and standard Bootstrap templates.

5.5 SECURITY DESIGN

Security protections include password hashing, Talisman CSP headers, Werkzeug hashing, and Redis-backed rate limiting to secure forms and prevent brute-force attacks.

5.6 MODULE DESIGN

The app is divided into distinct, logically separated blueprints: Auth Blueprint, Mood Blueprint, Admin Blueprint, ML Pipeline module, and NLP Sentiment analyzer.

6. IMPLEMENTATION

6.1 TECHNOLOGY STACK AND PROJECT STRUCTURE

The technology stack is carefully selected to support local data processing and hosting on student-friendly setups:

Technology	Description	Role inside DayTone
Python	Programming Language	Backend controllers, data modeling, ML, and NLP logic.
Flask	Web Framework	Routing, template rendering, and application factory creation.
PostgreSQL	Relational Database	Persistent storage in production environments.
SQLite	Relational Database	Local development database.

6.1 TECHNOLOGY STACK AND PROJECT STRUCTURE (CONTINUED)

Technology	Description	Role inside DayTone
Redis	In-memory Cache Store	Limiter registry, query caching, and token storage.
scikit-learn	ML Framework	Decision Tree / Random Forest models, feature calculation, and pipeline retraining.
NLTK VADER	NLP Lexicon Library	Local sentiment parsing of journal text.
Three.js WebGL	Graphics Library	Renders the interactive 3D orb using custom shaders.
Chart.js	Visualizations Library	Renders the analytics dashboards and goal milestones.

6.2 USER REGISTRATION AND LOGIN

User accounts are protected through password hashing (Werkzeug script) and CSRF protection. Admin registrations use secure invite tokens generated in the database.

[SCREENSHOT PLACEHOLDER]
Figure 4: User Registration and Login Screens
(Insert User Interface Screen Here)

Figure 4: User registration and secure login screen layout

6.3 DAILY LOGGING AND JOURNAL INTAKE

Users log their daily metrics (mood, sleep, stress, activity, and social indicators) alongside journal notes, which are encrypted using AES-256 before being written to the database.

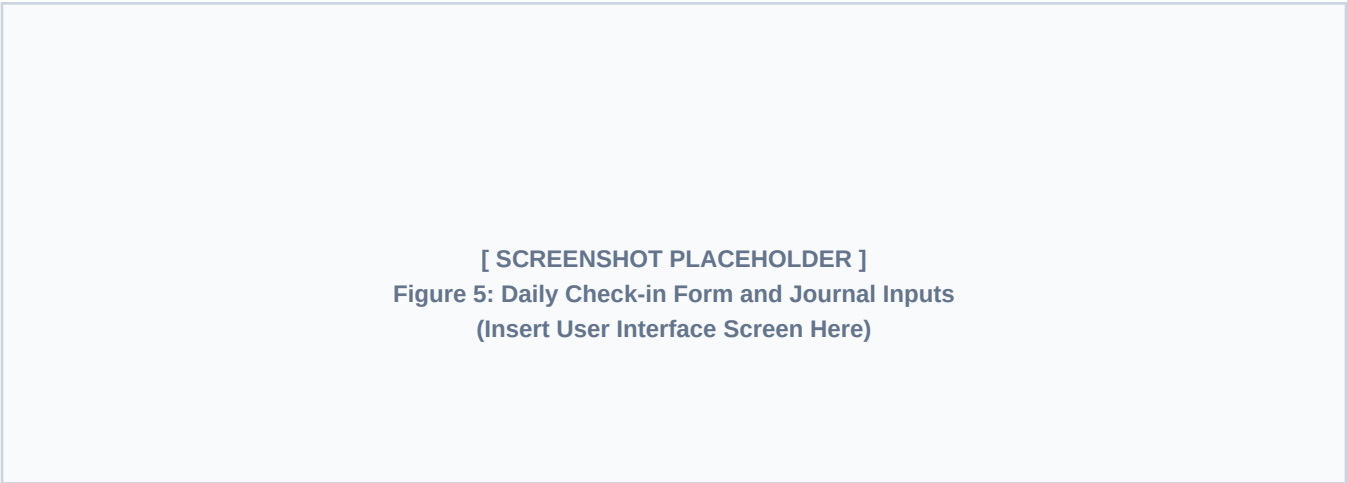


Figure 5: Daily check-in and encrypted journal submission form

6.4 WEBGL SHADER ORB RENDERING

The dashboard renders an interactive 3D orb using Three.js and custom vertex/fragment shaders. The orb deforms and changes color dynamically based on user metrics.



Figure 6: User dashboard showing mood trends and WebGL 3D orb

6.5 LOCAL SENTIMENT PARSING PIPELINE

To protect user privacy, DayTone processes sensitive journal text locally. By default, the platform initializes the NLTK VADER sentiment intensity analyzer. VADER evaluates the emotional valence of word tokens, outputting compound sentiment scores in `[-1.0, 1.0]`. If the server has sufficient hardware resources and the `transformers` library is detected, the application loads a lightweight DistilBERT pipeline. If both NLP pipelines fail (e.g. during initialization errors), the system falls back to a safe neutral score of 0.0, maintaining application availability.

6.6 MACHINE LEARNING BURNOUT MODEL INFERENCE

The machine learning system predicts burnout risk (Low, Medium, High) using a 12-feature matrix. This matrix captures both daily inputs and rolling 7-day averages. The predictor loads the scikit-learn Random Forest model from `model.pkl`. To prevent server downtime if the model file is missing or scikit-learn fails, the inference route falls back to a deterministic rule-based scoring engine.

6.7 WELLNESS RECOMMENDATION TIPS GENERATOR

A rule-based wellness engine checks daily metrics and displays customized suggestions. If a High burnout risk is predicted, emergency helpline contacts are displayed on the dashboard.

6.8 REPORT GENERATION AND EXPORT

Users can export their complete logging history as a CSV file or download a formatted PDF report generated using ReportLab. The PDF includes user information, daily logs, and trend charts.

6.9 ADMINISTRATIVE CONSOLE

Admins can manage the recommendation catalog, audit logs, and user accounts. All administrative actions are logged to ensure compliance and accountability.

[SCREENSHOT PLACEHOLDER]
Figure 7: Administrative Panel Screen
(Insert User Interface Screen Here)

Figure 7: Administrative interface displaying user catalogs and audit logs

6.10 WELLNESS SCORE BREAKDOWN

The platform breaks down the user's score across different criteria (sleep, mood, stress) and suggests specific tips to improve the score.

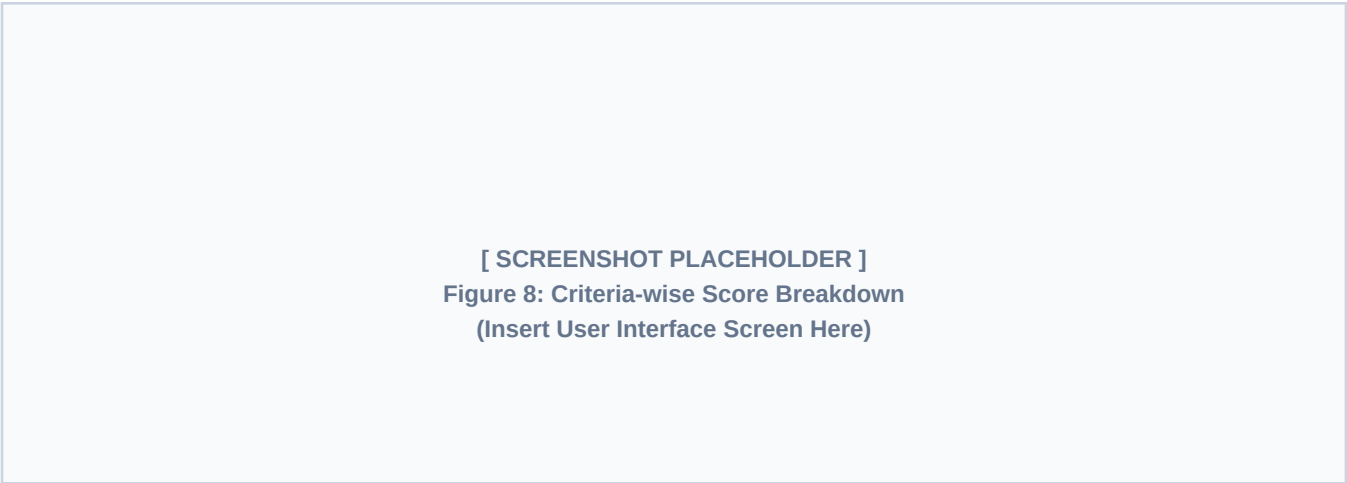


Figure 8: Criteria-wise wellness score breakdown and suggested tips

6.11 MODEL DIAGNOSTICS & RETRAINING

The administrative panel displays diagnostic charts tracking model performance. It includes bias audits and drift monitors to detect changes in accuracy.



Figure 9: Model diagnostics, bias reports and Kolmogorov-Smirnov drift charts

6.12 TESTING STRATEGY AND TEST CASE SUMMARY

Testing is performed using Pytest. Automated tests cover user registration, login, check-in, database encryption, model drift calculations, and admin catalog modifications.

Test Case ID	Component Verified	Expected Result
TC_01	User Registration & Auth	Creates account, hashes passwords, rejects duplicates.
TC_02	Daily Log Submission	Saves daily metrics, validates input ranges.
TC_03	Symmetric Note Encryption	Decrypts column value in memory, stores ciphertext.
TC_04	Local NLP Sentiment Parsing	Outputs sentiment score in standard range [-1, 1].
TC_05	ML Burnout Classification	Outputs Low/Medium/High, runs explainable AI drivers.
TC_06	Model Drift Calculations	Computes KS-test, raises warning if drift > 0.05.

7. RESULT & DISCUSSION

7.1 SAMPLE OUTPUT DESCRIPTION

The completed system successfully provides a browser-based wellness logging and predictive analysis workflow. Users can log daily metrics, review mood/sleep trends, check their burnout risk level, and view personalized wellness suggestions. High-risk predictions trigger crisis resources, which are hidden behind an opt-in reveal shield to manage user anxiety. The administrative panel displays diagnostic charts tracking model performance, bias reports, and Kolmogorov-Smirnov drift charts.

7.2 PERFORMANCE OBSERVATION

The system runs efficiently on standard hosts. Local sentiment scoring (VADER) and machine learning inference complete in under 100ms. If transformers are enabled, response latency increases slightly but remains under 1 second.

7.3 ADVANTAGES OF THE SYSTEM

DayTone provides a private, secure, and transparent mental wellness tracker. By implementing local NLP/ML engines, AES-256 column encryption, and PWA capabilities, the platform provides actionable wellness insights while keeping data secure in local memory.

7.4 LIMITATIONS

Key limitations include: VADER sentiment scoring is rule-based and lacks semantic understanding, and the scikit-learn classifier requires clean user logs to maintain prediction accuracy.

7.5 FUTURE ENHANCEMENTS

Future enhancements could include implementing semantic similarity using transformer-based embeddings, providing localized recommendations based on specific user profiles, and expanding multi-tenant organization features.

8. CONCLUSION

8.1 SUMMARY OF WORK COMPLETED

The DayTone Mood Analyser meets its goals of building a private, secure, and transparent mental wellness tracker. By implementing local NLP/ML engines, AES-256 column encryption, and PWA capabilities, the platform provides actionable wellness insights while keeping data secure in local memory.

8.2 LEARNING OUTCOMES

Key learning outcomes include implementing custom SQLAlchemy decorators, comparisons between Decision Tree and Random Forest classifiers, Three.js vertex shader programming, and GDPR-compliant cascading deletes.

APPENDIX A: PROJECT FILE STRUCTURE

Below is the listing of the DayTone codebase directory structure:

```
DayTone/
├── app/
│   ├── admin/
│   ├── auth/
│   ├── mood/
│   ├── ml/
│   ├── nlp/
│   ├── utils/
│   ├── templates/
│   ├── static/
│   ├── models.py
│   └── __init__.py
├── docs/
├── migrations/
├── scripts/
├── tests/
├── Dockerfile
└── requirements.txt

# Flask Application Package
# Administrative operations (auditing, user list)
# Sign-in/signup & GDPR purge views
# Check-in forms, dashboard & exports
# ML model, training, and predictor
# Sentiment parsing (VADER / DistilBERT)
# PDF reports, SOC-2 logs, & recommendations
# Jinja2 layout templates
# CSS/JS, WebGL orb, manifest, and service worker
# Database tables & Encryption properties
# App factory initialization
# Operations guides & runbooks
# Alembic database migration files
# Diagnostics (bias audit, drift check, CSS minifier)
# Pytest automation suite
# Production container definition
# Package dependencies
```

APPENDIX B: IMPORTANT ROUTES

Below are the primary Flask routes mapped across blueprints:

HTTP Route	Blueprint	Purpose / Description
/	auth_bp	Root URL. Serves the landing page or redirects to dashboard.
/login	auth_bp	Authenticates user sessions, redirects to dashboard.
/register	auth_bp	Creates user account, initializes profile fields.
/dashboard	mood_bp	Displays mood history, goal progress, and 3D orb UI.
/log	mood_bp	Accepts daily check-in forms and processes journals.
/export/pdf	mood_bp	Generates and downloads ReportLab PDF reports.
/admin/users	admin_bp	Admin user dashboard and profile control views.

APPENDIX C: IMPORTANT FUNCTIONS

Below are key backend function declarations and descriptions:

Function name	Module location	Objective
create_app()	app/__init__.py	Application factory method initializing extensions.
encrypt_text()	app/models.py	Encrypts text using Fernet AES-256 algorithm.
decrypt_text()	app/models.py	Decrypts Fernet encrypted strings in memory.
get_sentiment_score()	app/nlp/sentiment.py	Evaluates sentiment polarity using VADER / DistilBERT.
predict_burnout()	app/ml/predictor.py	Evaluates burnout risk utilizing Random Forest.
generate_pdf_report()	app/utlis/pdf_report.py	Compiles PDF wellness report exports.

APPENDIX D: REQUIREMENTS

Below are the contents of the requirements.txt configuration file:

```
Flask>=3.0.0
Flask-SQLAlchemy>=3.1.0
Flask-Login>=0.6.3
Flask-WTF>=1.2.1
Flask-Talisman>=1.1.0
Flask-Limiter>=3.5.0
Flask-Caching>=2.1.0
cryptography>=41.0.0
scikit-learn>=1.3.0
joblib>=1.3.0
pandas>=2.0.0
numpy>=1.24.0
nltk>=3.8.0
reportlab>=4.0.0
python-dotenv>=1.0.0
python-json-logger>=2.0.7
unicorn>=21.2.0
```

APPENDIX E: IMPORTANT CODE SNIPPETS

A. SQLAlchemy Models & Column mapping definitions in models.py

```
class User(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False, index=True)
    password_hash = db.Column(db.String(256), nullable=False)
    role = db.Column(db.String(20), nullable=False, default='user')
    created_at = db.Column(db.DateTime(timezone=True), default=utcnow, nullable=False)
    organization_id = db.Column(db.Integer, db.ForeignKey('organization.id', ondelete='SET NULL'))
    deleted_at = db.Column(db.DateTime(timezone=True), nullable=True)

class UserProfile(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('user.id', ondelete='CASCADE'), unique=True, nullable=False)
    preferred_activity = db.Column(db.String(50), nullable=False, default='Walk')
    calm_mode = db.Column(db.Boolean, default=False, nullable=False)
    predict_burnout = db.Column(db.Boolean, default=True, nullable=False)
```

APPENDIX E: IMPORTANT CODE SNIPPETS (CONTINUED)

B. Column-Level AES-256 Symmetric Encryption in models.py

```
def encrypt_text(value: str | None) -> str | None:
    if not value or _fernet is None:
        return value
    try:
        return _fernet.encrypt(value.encode('utf-8')).decode('ascii')
    except Exception:
        return value

def decrypt_text(value: str | None) -> str | None:
    if not value or _fernet is None:
        return value
    try:
        return _fernet.decrypt(value.encode('ascii')).decode('utf-8')
    except Exception:
        return value
```

APPENDIX E: IMPORTANT CODE SNIPPETS (CONTINUED)

C. Burnout Predictor & Rule Fallback in predictor.py

```
def predict_burnout(features: dict[str, float | int]) -> dict[str, str | float]:
    try:
        payload = _model_payload()
    except Exception as exc:
        payload = None

    if payload is None:
        prediction, confidence = _rule_prediction(features)
        return {
            "prediction": prediction,
            "confidence": confidence,
            "algorithm": "Rules",
            "drivers": explain_prediction(features, prediction)
        }
    model = payload["model"]
    values = pd.DataFrame([[features[name] for name in FEATURE_NAMES]], columns=FEATURE_NAMES)
    prediction = model.predict(values)[0]
    return {
        "prediction": prediction,
        "confidence": float(max(model.predict_proba(values)[0])),
        "algorithm": "RandomForest"
    }
```

APPENDIX E: IMPORTANT CODE SNIPPETS (CONTINUED)

D. Model Training & Pipeline in train.py

```
def train_model():
    data = pd.read_csv(TRAINING_DATA_PATH)
    X = data[FEATURE_NAMES]
    y = data['burnout_risk']
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    models = {
        'DecisionTree': DecisionTreeClassifier(max_depth=5, random_state=42),
        'RandomForest': RandomForestClassifier(n_estimators=100, max_depth=6, random_state=42)
    }
    best_acc = 0.0
    best_model = None
    for name, model in models.items():
        model.fit(X_train, y_train)
        acc = accuracy_score(y_test, model.predict(X_test))
        if acc > best_acc:
            best_acc = acc
            best_model = model

    joblib.dump({'model': best_model, 'features': FEATURE_NAMES}, MODEL_PATH)
```

APPENDIX E: IMPORTANT CODE SNIPPETS (CONTINUED)

E. Local Sentiment Score parsing in sentiment.py

```
def get_sentiment_score(text: str) -> float:
    if not text or not text.strip():
        return 0.0
    if _USE_HF and _hf_model is not None:
        try:
            result = _hf_model(text[:512])[0]
            return _hf_score_to_compound(result)
        except Exception:
            pass

    analyzer = _get_analyzer()
    if analyzer is None:
        return 0.0
    return float(analyzer.polarity_scores(text)['compound'])
```

APPENDIX E: IMPORTANT CODE SNIPPETS (CONTINUED)

F. Database Log Compiler in export_db_to_training.py

```
def compile_db_features(user_id):
    logs = MoodLog.query.filter_by(user_id=user_id).order_by(MoodLog.log_date.asc()).all()
    rows = []
    for log in logs:
        features = build_features(
            user_id=log.user_id,
            log_date=log.log_date,
            mood_score=log.mood_score,
            sleep_hours=log.sleep_hours,
            stress_level=log.stress_level,
            activity_done=log.activity_done,
            social_interaction=log.social_interaction,
            sentiment_score=log.sentiment_score
        )
        features['burnout_risk'] = log.burnout_risk
        rows.append(features)
    return pd.DataFrame(rows)
```

APPENDIX E: IMPORTANT CODE SNIPPETS (CONTINUED)

G. PyTest test cases in test_app.py

```
def test_mood_logging_flow(client, auth_headers):
    # 1. Post daily metrics check-in
    resp = client.post('/log', data={
        'mood_score': 4,
        'sleep_hours': 7.5,
        'stress_level': 2,
        'activity_done': 'y',
        'social_interaction': 3,
        'notes': 'Had a highly productive day, sleep was refreshing.'
    }, headers=auth_headers)
    assert resp.status_code == 302

    # 2. Verify encrypted field values and classification
    log = MoodLog.query.order_by(MoodLog.created_at.desc()).first()
    assert log.mood_score == 4
    assert log.notes == 'Had a highly productive day, sleep was refreshing.'
    assert log.burnout_risk == 'Low'
```

REFERENCES & BIBLIOGRAPHY

1. Flask Documentation, Pallets Projects, <https://flask.palletsprojects.com/>
2. Python Software Foundation Documentation, <https://docs.python.org/>
3. SQLite Consortium, File-based DB Engine, <https://www.sqlite.org/docs.html>
4. pandas Documentation, Data Handling structures, <https://pandas.pydata.org/docs/>
5. NumPy Developers, Numeric calculations, <https://numpy.org/doc/>
6. scikit-learn Machine Learning Library in Python, <https://scikit-learn.org/stable/>
7. NLTK Sentiment intensity, VADER Lexicon, <https://www.nltk.org/api/nltk.sentiment.html>
8. Three.js WebGL rendering engine, Three.js documentation, <https://threejs.org/docs/>
9. Chart.js visual graphics charting, <https://www.chartjs.org/docs/latest/>
10. ReportLab PDF Generation software, <https://www.reportlab.com/docs/>

REFERENCES & BIBLIOGRAPHY (CONTINUED)

11. PyTest testing framework automation, <https://docs.pytest.org/en/stable/>
12. Werkzeug Security & Utility helper, Pallets, <https://werkzeug.palletsprojects.com/>
13. Flask-Login session management, <https://flask-login.readthedocs.io/en/latest/>
14. Flask-WTF CSRF validation, <https://flask-wtf.readthedocs.io/en/latest/>
15. Flask-Talisman HTTP security headers, <https://github.com/GoogleCloudPlatform/flask-talisman>
16. Flask-Limiter token bucket rate logging, <https://flask-limiter.readthedocs.io/en/stable/>
17. Cryptography Fernet symmetric encryption, PyCA, <https://cryptography.io/en/latest/>
18. Evidently AI model drift calculations (KS-test), <https://docs.evidentlyai.com/>
19. Hugging Face Transformers pipeline (DistilBERT), <https://huggingface.co/docs/transformers/>
20. WCAG 2.1 Web Accessibility Guidelines, W3C Recommendation, <https://www.w3.org/TR/WCAG21/>